

Episodic AI

AI Powered Story Intelligence System for Short-Form Episodic Content

Abstract

With the rise of short-form video platforms such as reels and vertical storytelling platforms, creators increasingly produce episodic content consisting of short videos linked through a continuous narrative. However, many creators struggle with maintaining narrative flow, emotional engagement, and strong cliffhangers across episodes.

Episodic AI is an artificial intelligence powered storytelling assistant that analyzes story ideas and converts them into structured episodic narratives. The system evaluates storytelling quality using multiple AI modules including episode decomposition, emotional arc analysis, cliffhanger strength evaluation, and viewer retention prediction.

The system provides creators with a structured narrative breakdown, engagement insights, and optimization suggestions that help improve storytelling quality and audience retention.

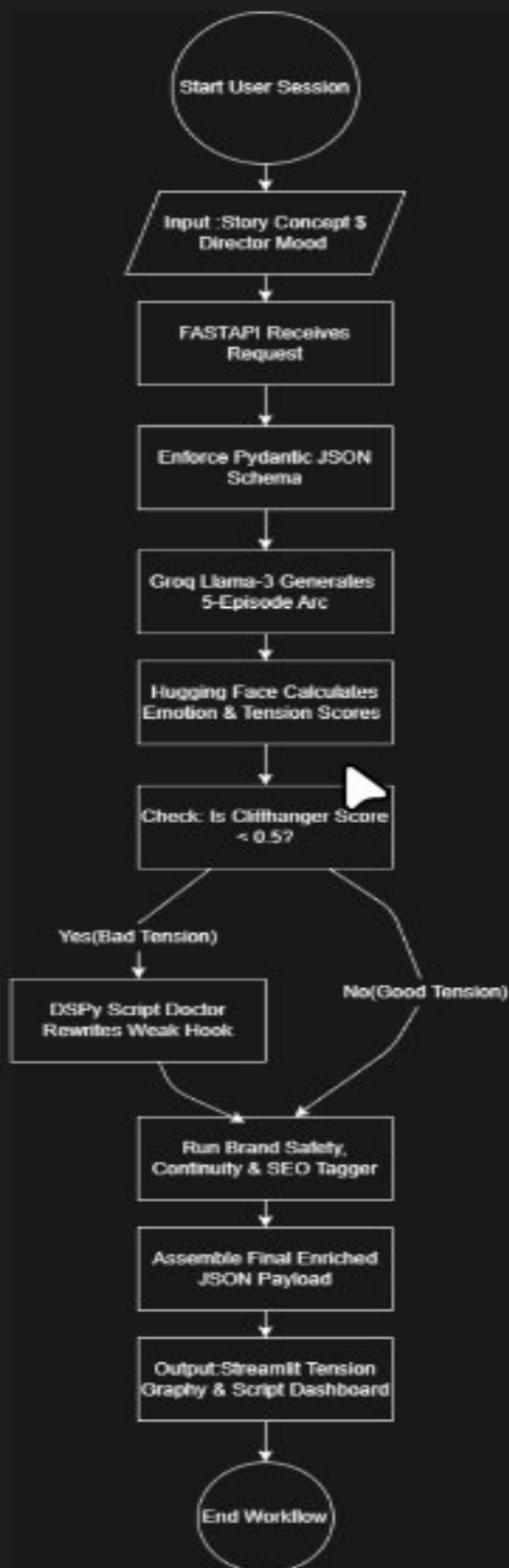
Problem Statement

Short-form storytelling platforms require content creators to produce engaging episodic narratives within very short time limits (typically under 90 seconds per episode).

However, creators face several challenges:

- Difficulty structuring a story into multiple engaging episodes
- Maintaining emotional progression across episodes
- Creating strong cliffhangers that encourage viewers to continue watching
- Predicting audience engagement and retention
- Improving pacing and narrative tension

Existing tools mainly generate scripts but do not provide **story analytics or narrative intelligence**.



Key Features

1. Story Decomposition

The system converts a single story idea into multiple short episodes.

Each episode includes:

- Title
- Summary
- Narrative progression
- Cliffhanger ending

2. Emotional Arc Analysis

The system detects emotional flow across episodes.

Example emotional arc:

Episode 1 → Curiosity

Episode 2 → Suspense

Episode 3 → Fear

Episode 4 → Shock

Episode 5 → Resolution

This helps creators maintain audience engagement.

3. Cliffhanger Strength Scoring

Each episode is evaluated for cliffhanger effectiveness.

Example:

Episode 2 Cliffhanger Score: 8/10

Reason:

- Unresolved mystery
- Character conflict introduced
- Unexpected reveal

This ensures viewers are encouraged to watch the next episode.

4. Retention Risk Prediction

The system predicts where viewers may lose interest.

Episodes are analyzed in time segments:

0–30 seconds

30–60 seconds

60–90 seconds

Example output:

Retention Risk:

0–30 sec → Low

30–60 sec → High (slow pacing)

60–90 sec → Medium

5. Optimization Suggestions

The AI system provides recommendations such as:

- Introduce conflict earlier
- Strengthen cliffhanger endings
- Improve emotional variation
- Add twists to maintain engagement

Character Continuity s Viral Packaging Engine

The Problem

In episodic storytelling, maintaining **character continuity and viewer engagement** across multiple episodes is a major challenge.

If characters behave inconsistently or episode titles fail to capture attention, viewers may lose interest in the narrative.

Additionally, short-form platforms such as:

- TikTok
- Instagram Reels
- YouTube Shorts

require **strong hooks and clickable titles** to capture audience attention within the first few seconds.

Therefore, the system must ensure:

- Story continuity across episodes
- Consistent narrative context
- High-engagement episode packaging

The Feature

The **Character Continuity s Viral Packaging Engine** ensures that each episode maintains narrative consistency while also generating engagement-optimized metadata.

This module automatically generates:

- Viral hooks
- Clickable episode titles
- Engagement-optimized metadata

These elements help maximize viewer retention and improve discoverability on social media platforms.

The Model

The system uses the **LLaMA-3-8B language model** for semantic extraction and storytelling optimization.

The model analyzes each episode summary and generates:

- A first-person viral hook
- A clickable episode title

This allows creators to package their stories effectively for short-form platforms.

How It Works

The system processes each episode in the generated story arc and applies AI-based semantic transformation.

Step 1 — Episode Summary Input

Each generated episode contains a summary describing the key events of the episode.

Example:

Episode Summary:

A student discovers a mysterious phone that predicts the future.

Step 2 — Semantic Analysis

The AI model analyzes the episode summary and identifies core narrative elements such as:

- Conflict
- Mystery
- Tension
- Discovery
- Danger

Step 3 – Viral Hook Generation

The system generates a **first-person viral hook** designed to trigger curiosity.

Example output:

Hook:

"I opened the phone... and it showed tomorrow's death."

Hook rules:

- Maximum 12 words
- First-person perspective
- Must create curiosity or tension

Step4 – Viral Episode Title Generation

The system also produces a short clickable title.

Example output:

Title:

"The Phone That Knows Tomorrow"

Title rules:

- Maximum 8 words
- Suspenseful tone
- Reflects the core conflict

Implementation Overview

The Character Continuity C Viral Packaging Engine processes each episode and enriches it with engagement-optimized metadata.

Key functions include:

`generate_viral_metadata()`

This function analyzes the episode summary and generates:

- Viral hook
- Episode title

`enhance_series_with_hooks()`

This function iterates through the episodic storyline and enriches each episode with viral metadata.

Example Output

Episode Structure Before Processing

Episode 2

Summary

The student tests the phone and realizes it predicts future events.

Episode Structure After Processing

Episode 2

Summary

The student tests the phone and realizes it predicts future events.

Viral Hook:

"I checked the phone... it predicted my accident."

Viral Title:

"The Phone That Knows My Future"

Technology Stack

Backend

- Python
- FastAPI

AI / NLP

- Large Language Models
- NLP emotion classification models
- Prompt engineering

AI Frameworks

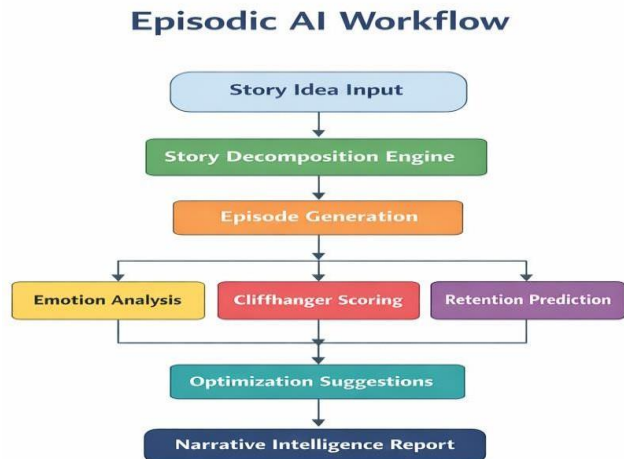
- HuggingFace Transformers
- Ollama (local LLM inference)
- Groq API (optional high-speed inference)

Frontend

- React / Web interface

Tools

- GitHub for version control
- REST APIs for system integration



AI Models Used

The Episodic AI system uses a combination of **local NLP models from the Hugging Face Transformers library** and **Large Language Models (LLMs)** for generation, reasoning, and narrative optimization.

These models power different components of the storytelling intelligence pipeline.

1. Hugging Face Models (Local Inference)

Several Natural Language Processing models are loaded locally using the **transformers pipeline**.

These models perform analytical tasks such as **emotion detection**, **narrative evaluation**, and **content safety checks**.

Emotion Classification Model

Model:

`j-hartmann/emotion-english-distilroberta-base`

Location in Project:

`analytics.py`

Purpose:

This model is used to detect the emotional tone of each episode.

- It classifies text into multiple emotion categories such as:

- Joy
- Fear
- Anger
- Sadness
- Surprise
- Neutral

Role in the System:

The emotion classification helps generate an **emotional arc across episodes**, which is critical for maintaining viewer engagement in episodic storytelling.

Example:

Episode Summary:

"The student realizes the phone predicts future disasters."

Detected Emotion:

Fear / Suspense

This information contributes to the **Emotional Arc Analysis Module**.

• **Cliffhanger Evaluation Model**

Model:

facebook/bart-large-mnli

Location in Project:

`analytics.py`

Purpose:

This model performs **zero-shot classification** to evaluate the strength of an episode's cliffhanger.

How It Works:

The model analyzes the episode summary and cliffhanger text to determine whether the ending contains narrative elements such as:

- suspense
- unresolved conflict
- mystery
- danger

- emotional tension

These signals are used to compute a **Cliffhanger Score**.

Example Output:

Cliffhanger Score:
0.82

Interpretation:

High probability that the episode ending encourages viewers to watch the next episode.

• **Brand Safety Model**

Model:

unitary/toxic-bert

Location in Project:

`brand_safety.py`

Purpose:

This model is used to ensure that generated content remains safe and appropriate for public platforms.

How It Works:

The model analyzes the combined text of:

- episode title
- episode summary
- cliffhanger

It detects potential toxic or unsafe language.

If the toxicity score exceeds a predefined threshold (e.g., 0.7), the episode is flagged.

Example Output:

is_brand_safe: False
safety_warning: "Potentially toxic content detected."

This module ensures the content remains suitable for platforms such as:

- TikTok
- Instagram Reels
- YouTube Shorts

2. Large Language Models (LLMs)

- Large Language Models are used for **story generation, narrative reasoning, and story improvement suggestions.**
- These models are accessed via the **Groq API.**

Story Generation Model

- Model:
`llama-3.1-8b-instant`

- Location in Project:
`generator.py`

- Purpose:

This model acts as the **primary narrative engine** of the system.

- Responsibilities:
 - Convert a user story idea into a **5-episode narrative arc**
 - Maintain consistent character context
 - Generate structured episodic storytelling output
 - Produce strict JSON formatted responses

- Example Generated Output:

```
{
  "episode": 1,
  "title": "The Phone Appears",
  "summary": "A college student finds a mysterious phone that predicts tomorrow's events.",
  "cliffhanger": "The phone shows tomorrow's headline: 'Student Dies in Accident'."
}
```

This model forms the **core storytelling generator** of Episodic AI.

Script Doctor Model

- Model:
`llama-3.1-8b-instant`

- Location in Project:
`script_doctor.py`

- Purpose:

This model is used within the **DSPy framework** to improve weak narrative segments.

Functionality:

The Script Doctor analyzes episodes with **low cliffhanger scores** and generates a short, actionable suggestion to improve story tension.

- Inputs:
 - episode_summary
 - cliffhanger_score
- Example Feedback:

"Increase the stakes by revealing that the phone predicts the protagonist's own death."
- Key Characteristics:
 - Produces concise narrative improvement suggestions
 - Helps strengthen episode endings
 - Enhances storytelling tension

Sample Input

Story Idea:

A college student finds a phone that can predict future events.

Sample Output

Episode Breakdown

Episode 1 - The Discovery

A student discovers a mysterious phone that shows tomorrow's messages.

Cliffhanger:

The phone predicts an accident happening the next day.

Episode 2 - The Warning

The student tries to prevent the accident.

Cliffhanger Score: 8/10

Emotional Arc

Curiosity → Suspense → Fear → Shock → Resolution

Retention Analysis

Episode 2

0-30 sec → Strong hook

30-60 sec → Medium engagement

60-90 sec → Strong tension

AI Suggestions

- Introduce a secondary character earlier.
- Strengthen Episode 3 twist.
- Increase emotional conflict in Episode 2.

Advantages

- Helps creators structure episodic stories
- Improves viewer engagement
- Provides storytelling analytics
- Enhances narrative quality
- Useful for short-form content creators

Applications

Episodic AI can be used in:

- Short-video content creation
- Web series planning
- Script writing assistance
- Social media storytelling
- Creative writing education
- Film and media production

Future Enhancements

Future improvements may include:

- Real viewer analytics integration
- Automatic script generation
- Visual storyboarding tools

- Character consistency tracking
- Multi-language storytelling support
- Integration with video creation tools

Conclusion

Episodic AI demonstrates how artificial intelligence can assist creators in designing more engaging episodic narratives. By combining story decomposition, emotional analysis, cliffhanger scoring, and retention prediction, the system acts as an intelligent storytelling assistant.

The platform helps creators transform raw story ideas into structured episodic content while improving narrative quality and viewer engagement.